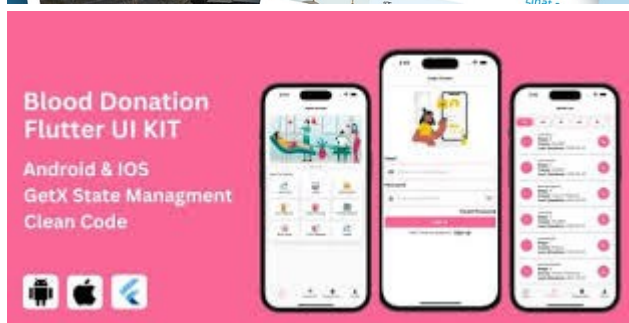
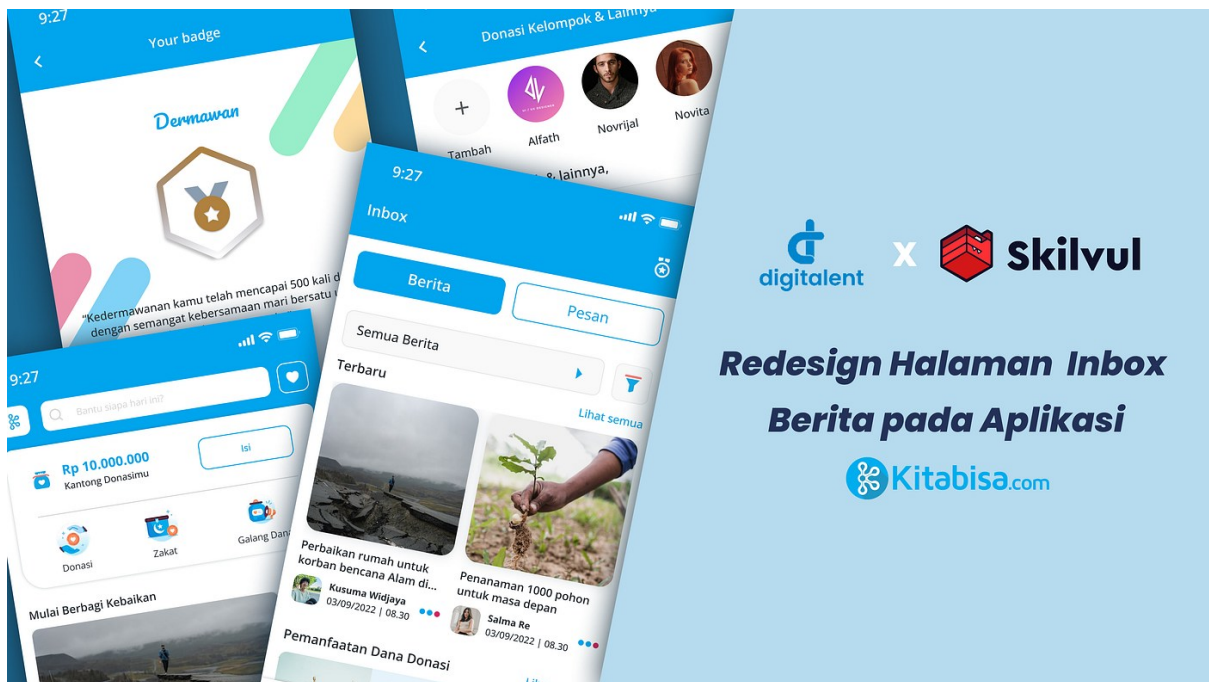
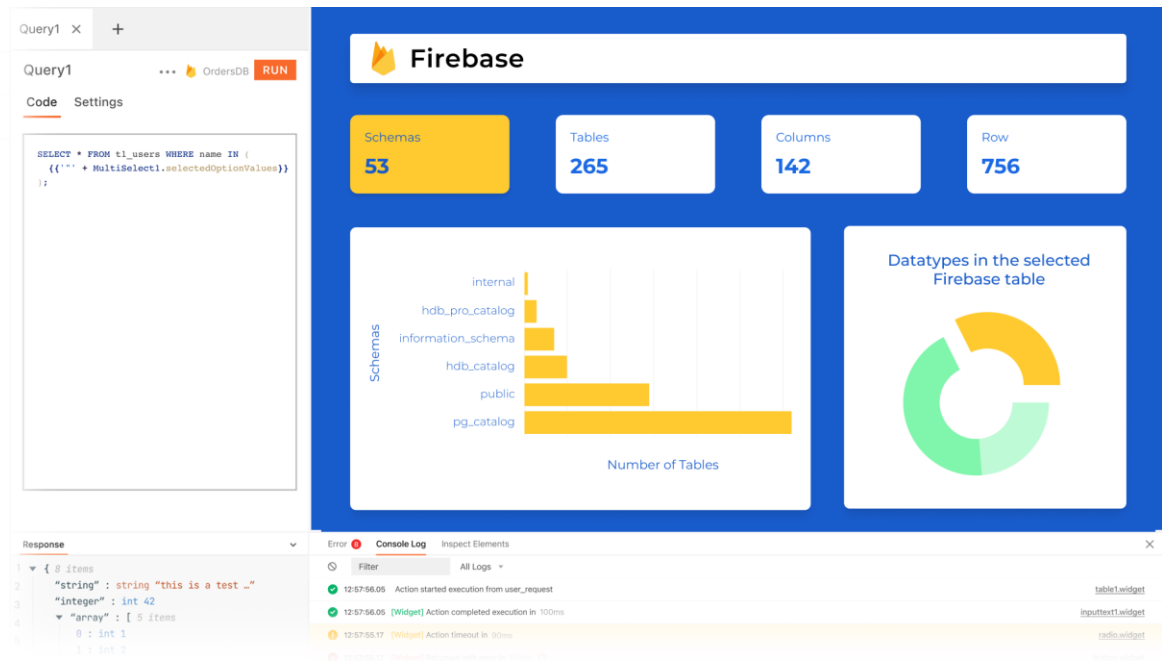


MATERI AJAR PRAKTIK

Membangun Aplikasi Donasi Online Berbasis Flutter & Firebase

1. Gambaran Aplikasi Donasi Online





4

Aplikasi donasi online adalah platform mobile yang memungkinkan pengguna untuk:

- Melihat kampanye donasi
- Memberikan donasi
- Mengelola data donasi secara real-time

🎯 Fitur Utama:

- 🔒 Login & Register (Firebase Auth)
- 📊 Dashboard Kampanye Donasi
- ➕ Tambah Kampanye (Admin)
- 💰 Donasi ke Kampanye
- 📜 Riwayat Donasi
- 🖼️ Upload Gambar (Firebase Storage)

2. 🎓 Capaian Pembelajaran

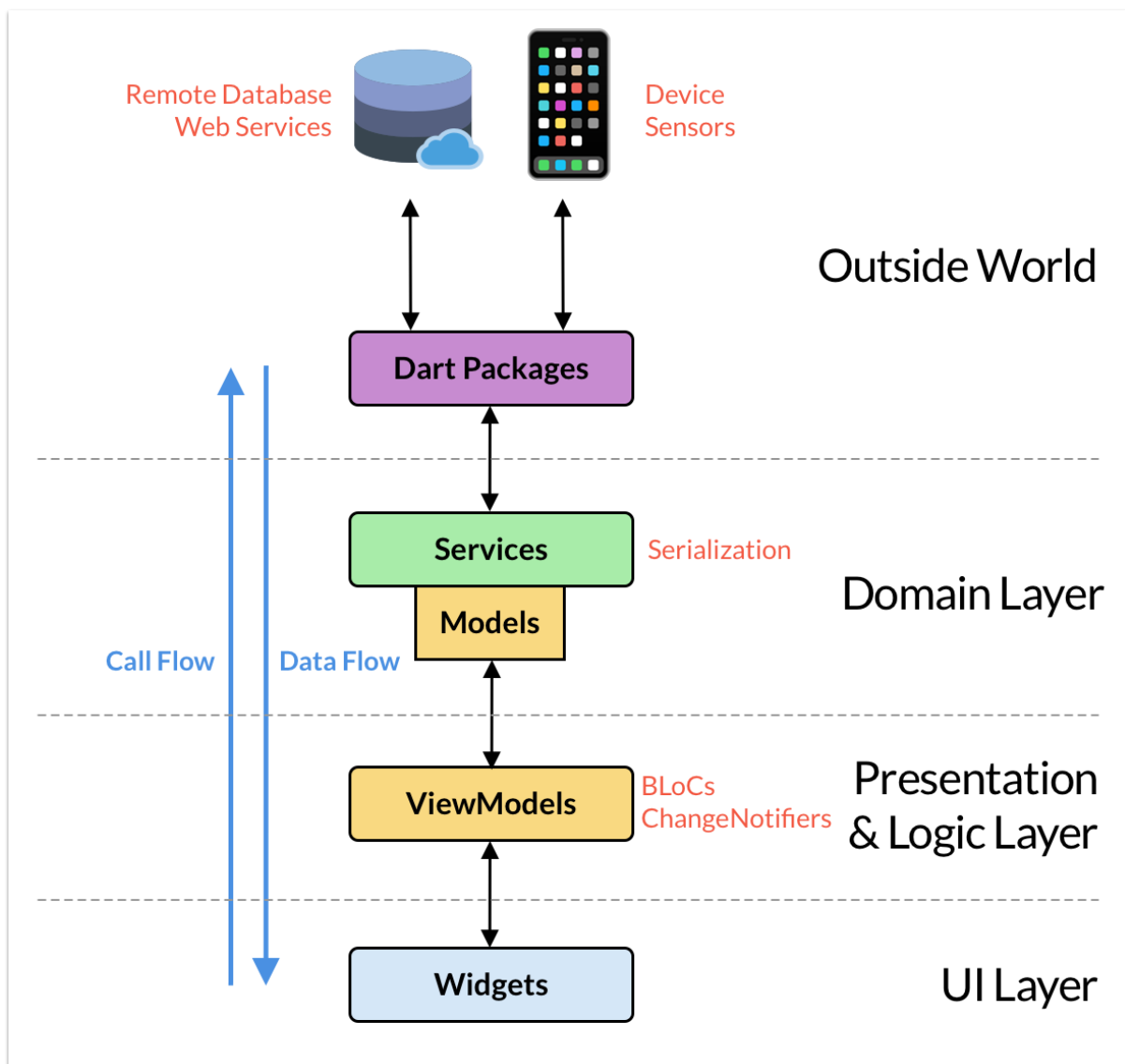
Mahasiswa mampu:

- Mengembangkan aplikasi Flutter berbasis Firebase
- Mengimplementasikan autentikasi user
- Mengelola database NoSQL (Firestore)
- Mengintegrasikan storage cloud
- Membangun aplikasi real-time

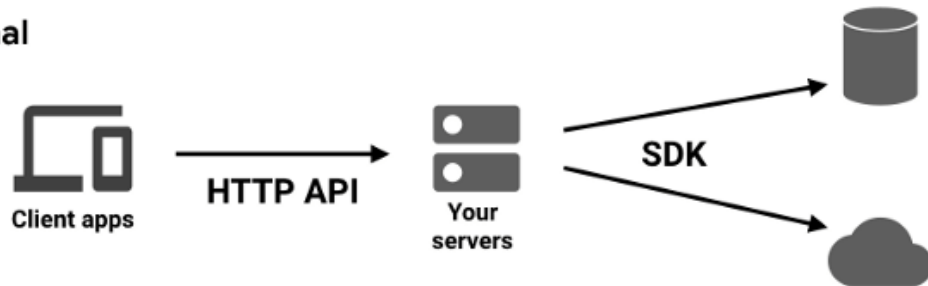
3. 🛠️ Tools & Teknologi

- Flutter SDK
 - Dart
 - Firebase:
 - Authentication
 - Cloud Firestore
 - Firebase Storage
-

4. 🏗️ Arsitektur Sistem



Traditional



Firebase



4

Penjelasan:

- Flutter sebagai frontend
- Firebase sebagai backend (BaaS)
- Firestore untuk database
- Storage untuk gambar
- Authentication untuk login

5. Setup Project

A. Buat Project Flutter

```
flutter create donasi_app  
cd donasi_app
```

B. Tambahkan Dependency

dependencies:

flutter:

 sdk: flutter

 firebase_core: ^2.0.0

```
firebase_auth: ^4.0.0
cloud_firestore: ^4.0.0
firebase_storage: ^11.0.0
image_picker: ^1.0.0
```

6. Implementasi Login & Register

Struktur Koleksi User (Firestore)

```
users
├── userId
│   ├── nama
│   └── email
```

Contoh Coding Register:

```
Future<void> register(String email, String password) async {
  await FirebaseAuth.instance.createUserWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

Contoh Login:

```
Future<void> login(String email, String password) async {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

7. Dashboard Kampanye Donasi

07:31



MES ÉLÈVES



1 Aba Jack



Sixième 3



- Ajouté(e) le 31 mars 2022
- Aucune alerte signalée

MODIFIER
NOM

AFFECTER
CLASSE

SUPPRIMER

ENVOYER
ACTIVITÉ

ENVOYER DÉFI

VOIR BILAN
GÉNÉRAL

LES ACTIVITÉS



LES DÉFIS



2 Dupont André



Sixième 3

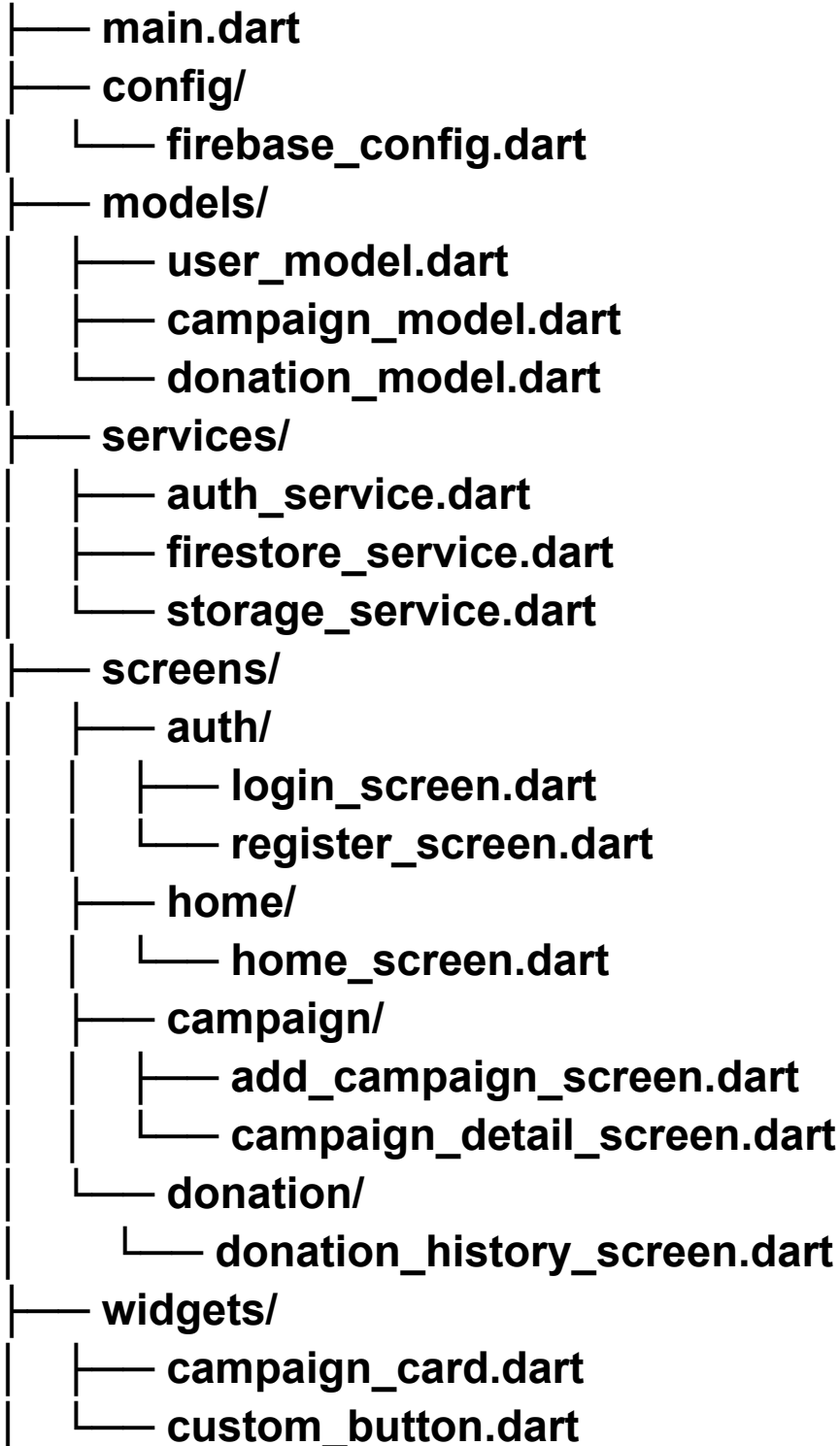


- Ajouté(e) le 31 mars 2022
- Aucune alerte signalée



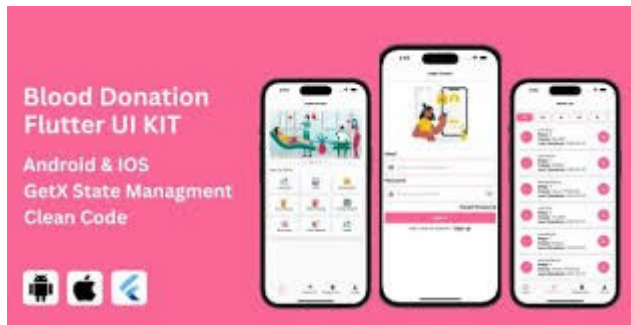
1. Struktur Project (Best Practice Flutter)

lib/

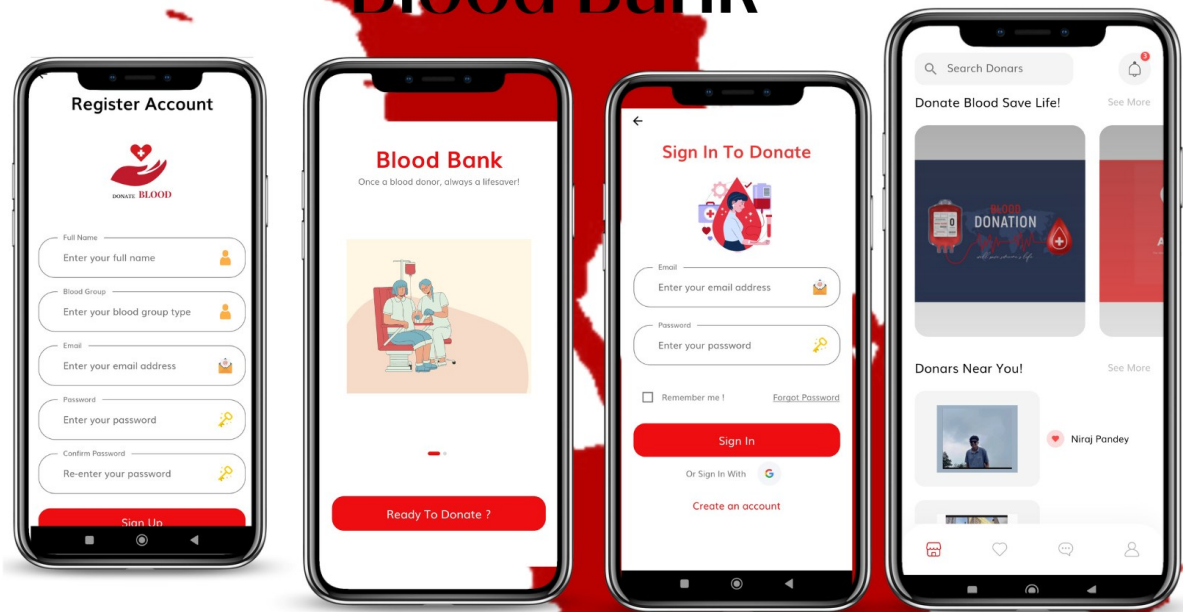




2. Gambaran UI Aplikasi



Blood Bank



4



3. main.dart (Inisialisasi Firebase)

```
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'screens/auth/login_screen.dart';
```

```
void main() async {
```



```
WidgetsFlutterBinding.ensureInitialized();
await Firebase.initializeApp();
runApp(MyApp());
}
```

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: LoginScreen(),
    );
  }
}
```



4. Auth Service

```
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<User?> login(String email, String password)
  async {
    var result = await
    _auth.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
    return result.user;
  }
}
```

```
}
```

```
Future<User?> register(String email, String password)
async {
  var result = await
_auth.createUserWithEmailAndPassword(
  email: email,
  password: password,
);
  return result.user;
}
```

```
Future<void> logout() async {
  await _auth.signOut();
}
}
```

5. Firestore Service

```
import 'package:cloud_firestore/cloud_firestore.dart';

class FirestoreService {
  final FirebaseFirestore _db =
  FirebaseFirestore.instance;

  Stream<QuerySnapshot> getCampaigns() {
    return _db.collection('campaigns').snapshots();
  }
}
```

```
Future<void> addCampaign(Map<String, dynamic>
data) async {
  await _db.collection('campaigns').add(data);
}
```

```
Future<void> donate(String id, int amount) async {
  var ref = _db.collection('campaigns').doc(id);
```

```
  await _db.runTransaction((transaction) async {
    var snapshot = await transaction.get(ref);
    int newTotal = snapshot['collected'] + amount;
    transaction.update(ref, {'collected': newTotal});
  });
}
```

6. Model (Contoh Campaign)

```
class Campaign {
  String id;
  String title;
  String description;
  int target;
  int collected;
  String imageUrl;

  Campaign({
    required this.id,
    required this.title,
    required this.description,
```

```
required this.target,  
required this.collected,  
required this.imageUrl,  
});  
  
factory Campaign.fromFirestore(doc) {  
  return Campaign(  
    id: doc.id,  
    title: doc['title'],  
    description: doc['description'],  
    target: doc['target'],  
    collected: doc['collected'],  
    imageUrl: doc['imageUrl'],  
  );  
}  
}
```



7. Home Screen (Dashboard)

```
StreamBuilder(  
  stream: FirestoreService().getCampaigns(),  
  builder: (context, snapshot) {  
    if (!snapshot.hasData) return  
    CircularProgressIndicator();  

```

```
    var campaigns = snapshot.data!.docs;
```

```
    return ListView.builder(  
      itemCount: campaigns.length,  
      itemBuilder: (context, index) {
```

```

var data = campaigns[index];

return Card(
  child: ListTile(
    leading: Image.network(data['imageUrl']),
    title: Text(data['title']),
    subtitle: Text(
      "Terkumpul: ${data['collected']} /
    ${data['target']}"
    ),
  ),
);
},
);
},
)

```

+ 8. Tambah Campaign + Upload Gambar

```

import 'dart:io';
import
'package:firebase_storage/firebase_storage.dart';

Future<String> uploadImage(File file) async {
  var ref = FirebaseStorage.instance
    .ref()
    .child('campaign/${DateTime.now()}.jpg');

  await ref.putFile(file);

```

```
return await ref.getDownloadURL();  
}
```

9. Halaman Donasi

```
ElevatedButton(  
  onPressed: () async {  
    await FirestoreService().donate(campaignId, 50000);  
  },  
  child: Text("Donasi 50.000"),  
)
```

10. Riwayat Donasi (Firestore)

Struktur:

```
donations/  
├── id  
│   ├── userId  
│   ├── campaignId  
│   ├── amount  
│   └── date
```

11. Integrasi Payment Gateway (Opsional Lanjut)

Midtrans (Best Practice Indonesia)

Tambahkan:

- Snap API
- WebView Flutter

Alur:

1. User klik donasi
 2. Generate transaksi di backend
 3. Redirect ke payment gateway
 4. Callback update Firestore
-



12. Notifikasi (FCM)

Gunakan:

firebase_messaging: ^14.0.0

Fungsi:

- Notifikasi campaign baru
 - Notifikasi donasi berhasil
-



13. Tugas Project Mahasiswa

Wajib:

- Login & Register
- Dashboard Campaign
- Donasi real-time

Lanjutan:

- Upload gambar

- Riwayat donasi
- Role admin/user

Challenge (Sangat Direkomendasikan untuk Skripsi/Q1):

- AI rekomendasi donasi
 - Fraud detection
 - Blockchain transparency
-

14. Insight Akademik (Nilai Tambah Penelitian)

Topik ini bisa dikembangkan menjadi:

- Smart Charity System
- Fintech berbasis sosial
- Integrasi AI + Flutter + Cloud



Track Order

CANCEL

ESTIMATED TIME

30 minutes

ORDER NUMBER

#2482011



Order Placed

We have received your order.



Order Confirmed

Your order has been confirmed.



Order Processed

We are preparing your order.



Ready to Pickup

Your order is ready for pickup.

Struktur Data Firestore:

```

campaigns
├── campaignId
│   ├── title
│   ├── description
│   ├── target
│   ├── collected
│   └── imageUrl

```

Contoh Ambil Data:

```

StreamBuilder(
  stream: FirebaseFirestore.instance.collection('campaigns').snapshots(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) return CircularProgressIndicator();
    var data = snapshot.data!.docs;
    return ListView.builder(
      itemCount: data.length,
      itemBuilder: (context, index) {
        return ListTile(
          title: Text(data[index]['title']),
          subtitle: Text(data[index]['description']),
        );
      },
    );
  },
);

```

8. + Tambah Kampanye + Upload Gambar

Upload Gambar ke Firebase Storage:

```

Future<String> uploadImage(File image) async {
  var ref = FirebaseStorage.instance
    .ref()
    .child('campaign_images/${DateTime.now()}.jpg');
  await ref.putFile(image);
  return await ref.getDownloadURL();
}

```

Simpan ke Firestore:

```

Future<void> addCampaign(String title, String desc, String imageUrl) async {
  await FirebaseFirestore.instance.collection('campaigns').add({
    'title': title,

```

```
'description': desc,  
'target': 1000000,  
'collected': 0,  
'imageUrl': imageUrl,  
});  
}
```

9. 💰 Fitur Donasi

Update Donasi:

```
Future<void> donate(String campaignId, int amount) async {  
  var doc = FirebaseFirestore.instance.collection('campaigns').doc(campaignId);  
  
  await FirebaseFirestore.instance.runTransaction((transaction) async {  
    var snapshot = await transaction.get(doc);  
    int newAmount = snapshot['collected'] + amount;  
  
    transaction.update(doc, {'collected': newAmount});  
  });  
}
```

10. 📄 Riwayat Donasi

Struktur:

```
donations  
├── donationId  
│   ├── userId  
│   ├── campaignId  
│   ├── amount  
│   └── date
```

11. 🔄 Integrasi Real-Time

Firestore memungkinkan:

- Update otomatis tanpa refresh
 - Sinkronisasi multi user
-

12. 🧪 Tugas Praktikum

Level Dasar:

- Implementasi login & register

Level Menengah:

- CRUD campaign

Level Lanjut:

- Sistem donasi + riwayat

Challenge:

- Integrasi payment gateway (Midtrans/Xendit)
 - Notifikasi (Firebase Cloud Messaging)
-

13. 📊 Evaluasi

Komponen	Bobot
UI/UX	20%
Fungsionalitas	30%
Integrasi Firebase	30%
Presentasi	20%

14. 🚀 Pengembangan Lanjutan

- Role Admin & User
 - Verifikasi donasi
 - AI rekomendasi campaign
 - Blockchain untuk transparansi donasi
-

15. 📌 Kesimpulan

Aplikasi ini menggabungkan:

- Mobile Development (Flutter)
- Cloud Backend (Firebase)
- Real-time system

Sehingga sangat relevan untuk:

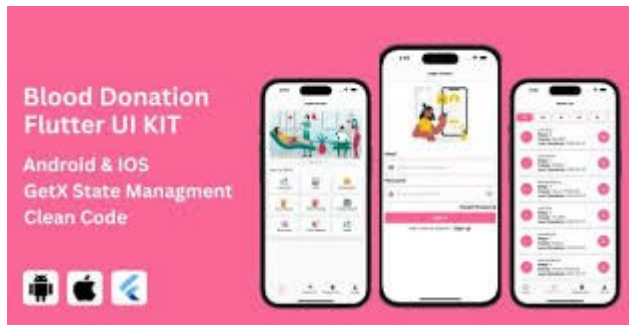
- Startup digital
- Smart charity platform
- Proyek skripsi / penelitian IoT + AI

🚀 1. Struktur Project (Best Practice Flutter)

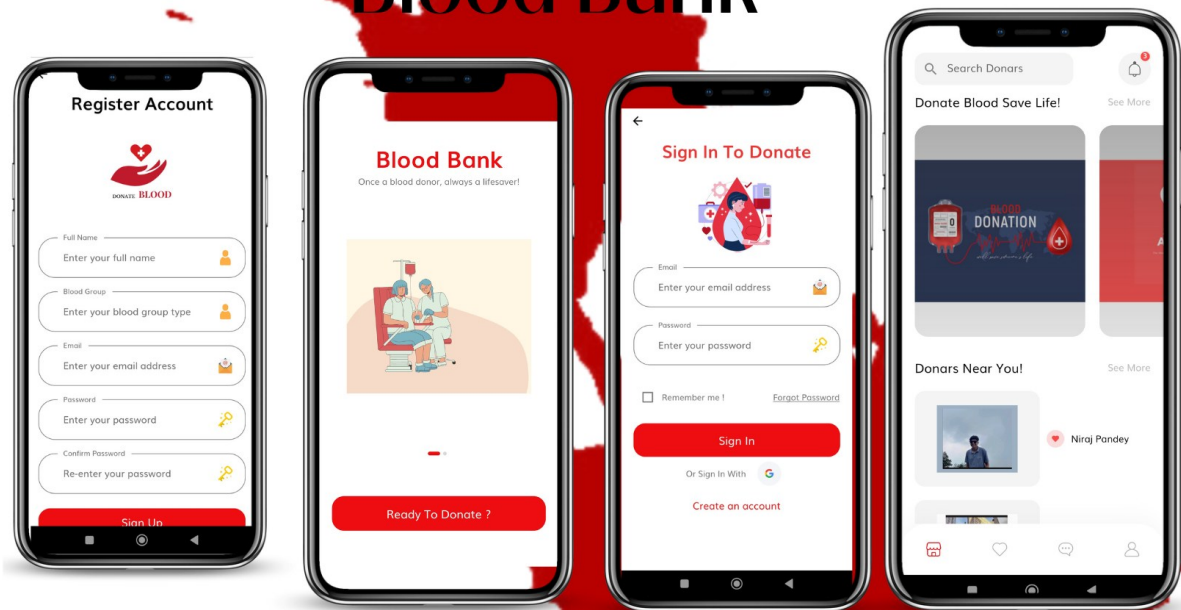
```
lib/
├── main.dart
├── config/
│   └── firebase_config.dart
├── models/
│   ├── user_model.dart
│   ├── campaign_model.dart
│   └── donation_model.dart
├── services/
│   ├── auth_service.dart
│   ├── firestore_service.dart
│   └── storage_service.dart
├── screens/
│   ├── auth/
│   │   ├── login_screen.dart
│   │   └── register_screen.dart
│   ├── home/
│   │   └── home_screen.dart
│   ├── campaign/
│   │   ├── add_campaign_screen.dart
│   │   └── campaign_detail_screen.dart
│   └── donation/
│       └── donation_history_screen.dart
├── widgets/
│   ├── campaign_card.dart
│   └── custom_button.dart
```



2. Gambaran UI Aplikasi



Blood Bank



4



3. main.dart (Inisialisasi Firebase)

```
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'screens/auth/login_screen.dart';
```

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp();  
  runApp(MyApp());  
}
```

```
}  
  
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: LoginScreen(),  
    );  
  }  
}
```



4. Auth Service

```
import 'package:firebase_auth/firebase_auth.dart';  
  
class AuthService {  
  final FirebaseAuth _auth = FirebaseAuth.instance;  
  
  Future<User?> login(String email, String password) async {  
    var result = await _auth.signInWithEmailAndPassword(  
      email: email,  
      password: password,  
    );  
    return result.user;  
  }  
  
  Future<User?> register(String email, String password) async {  
    var result = await _auth.createUserWithEmailAndPassword(  
      email: email,  
      password: password,  
    );  
    return result.user;  
  }  
  
  Future<void> logout() async {  
    await _auth.signOut();  
  }  
}
```



5. Firestore Service

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```

class FirestoreService {
    final FirebaseFirestore _db = FirebaseFirestore.instance;

    Stream<QuerySnapshot> getCampaigns() {
        return _db.collection('campaigns').snapshots();
    }

    Future<void> addCampaign(Map<String, dynamic> data) async {
        await _db.collection('campaigns').add(data);
    }

    Future<void> donate(String id, int amount) async {
        var ref = _db.collection('campaigns').doc(id);

        await _db.runTransaction((transaction) async {
            var snapshot = await transaction.get(ref);
            int newTotal = snapshot['collected'] + amount;
            transaction.update(ref, {'collected': newTotal});
        });
    }
}

```



6. Model (Contoh Campaign)

```

class Campaign {
    String id;
    String title;
    String description;
    int target;
    int collected;
    String imageUrl;

    Campaign({
        required this.id,
        required this.title,
        required this.description,
        required this.target,
        required this.collected,
        required this.imageUrl,
    });

    factory Campaign.fromFirestore(doc) {
        return Campaign(
            id: doc.id,

```



```

        title: doc['title'],
        description: doc['description'],
        target: doc['target'],
        collected: doc['collected'],
        imageUrl: doc['imageUrl'],
    );
}
}

```



7. Home Screen (Dashboard)

```

StreamBuilder(
  stream: FirestoreService().getCampaigns(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) return CircularProgressIndicator();

    var campaigns = snapshot.data!.docs;

    return ListView.builder(
      itemCount: campaigns.length,
      itemBuilder: (context, index) {
        var data = campaigns[index];

        return Card(
          child: ListTile(
            leading: Image.network(data['imageUrl']),
            title: Text(data['title']),
            subtitle: Text(
              "Terkumpul: ${data['collected']} / ${data['target']}",
            ),
          ),
        );
      },
    );
  },
)

```

+ 8. Tambah Campaign + Upload Gambar

```

import 'dart:io';
import 'package:firebase_storage/firebase_storage.dart';

```

```
Future<String> uploadImage(File file) async {
  var ref = FirebaseStorage.instance
    .ref()
    .child('campaign/${DateTime.now()}.jpg');

  await ref.putFile(file);
  return await ref.getDownloadURL();
}
```



9. Halaman Donasi

```
ElevatedButton(
  onPressed: () async {
    await FirestoreService().donate(campaignId, 50000);
  },
  child: Text("Donasi 50.000"),
)
```



10. Riwayat Donasi (Firestore)

Struktur:

```
donations/
├── id
│   ├── userId
│   ├── campaignId
│   ├── amount
│   └── date
```



11. Integrasi Payment Gateway (Opsional Lanjut)

Midtrans (Best Practice Indonesia)

Tambahkan:

- Snap API

- WebView Flutter

Alur:

1. User klik donasi
 2. Generate transaksi di backend
 3. Redirect ke payment gateway
 4. Callback update Firestore
-



12. Notifikasi (FCM)

Gunakan:

firebase_messaging: ^14.0.0

Fungsi:

- Notifikasi campaign baru
 - Notifikasi donasi berhasil
-



13. Tugas Project Mahasiswa

Wajib:

- Login & Register
- Dashboard Campaign
- Donasi real-time

Lanjutan:

- Upload gambar
- Riwayat donasi
- Role admin/user

Challenge (Sangat Direkomendasikan untuk Skripsi/Q1):

- AI rekomendasi donasi
 - Fraud detection
 - Blockchain transparency
-

14. Insight Akademik (Nilai Tambah Penelitian)

Topik ini bisa dikembangkan menjadi:

- Smart Charity System
- Fintech berbasis sosial
- Integrasi AI + Flutter + Cloud